ÍNDICE

1 Conceptos a repasar..... 1
2 Programando hilos..... 2

1 Conceptos a repasar

- Thread vs Proceso. Todo proceso tiene al menos un hilo. Múltiples hilos de un proceso comparten contexto.
- Programación secuencial. Programación concurrente.
- Método start vs método run.
- Clase Thread vs Interfaz Runnable.
- Estado hilos. Nuevo (New), Ejcutable (Runnable), Bloqueado (Blocked), Esperando (Waiting), Esperando tiempo (Time_waiting), Finalizado (Terminated). Ver foto en apuntes Inaki.
- Clases con concurrencia de hilos de java.lang. Interfaz Runnable, clase Thread, clase Timer.
- Uso de Thread.sleep(int i).
- Método interrupt de la clase Thread. Excepción Interrupted Exception capturada para Thread.sleep().
- Multitarea, Recursos compartidos (monitor), accesos/operación atómicos, sección crítica, exclusión mutua.
- Posibles problemas. Interbloqueo, inanición.
- join, Shyncronized, wait - notify - notifyall, semaphore.
- Métodos de la clase Thread. run, interrupt, isInterrupted, sleep, setName, getName, yield (equivalente a Thread.sleep(0)), isAlive, setPriority (0-10 tb Thread.MIN_PRIORITY, Thread.NORMAL_PRIORITY, Thread.MAX_PRIORITY).
- Ejercicio tipo productor-consumidor.
- Semaphore. Métodos típicos. acquire, availablePermits, release, isFair.

2 Programando hilos

Crear hilo con herencia de Thread

```
public class T2RatonesThr extends Thread {

	public static void main (String [] args)
	{
		T2RatonesThr miTh1 = new T2RatonesThr();
		miTh1.start();
		T2RatonesThr miTh2 = new T2RatonesThr();
		miTh2.start();
	}
	@Override
	public void run ()
	{
		System.out.println("HOLA");
	}
}
```

Crear hilo implementando el interfaz Runnable.

Si hay varios núcleos con iteraciones altas: exc. Mutua

```
public class T2RatonesRun  implements Runnable {
	int iValor = 0;
	public static void main (String [] args)
	{
		T2RatonesRun miRat = new T2RatonesRun();

		for(int i=0;i<10000;i++)
		{
			new Thread(miRat).start();
		}

		try {
			Thread.sleep(10);
		} catch (InterruptedException e) {
			e.printStackTrace();
		}
		System.out.println(miRat.iValor);
	}

	@Override
	public void run() {
		iValor++;
	}
}
```

Estados de un hilo

```
import java.util.ArrayList;

// Los estados del hilo pueden ser	Nuevo, Ejecutable, Bloqueado, Esperando, Esperando un tiempo, Finalizado
// En Thread.State aparecen como NEW, RUNNABLE, BLOCKED, WAITING, TIME_WAITING, TERMINATED
public class T2EstadoHilos implements Runnable {
	public static void main (String [] args)
	{
		T2EstadoHilos miEstHi = new T2EstadoHilos();
		ArrayList <Thread.State> miArrEs = new ArrayList<Thread.State>();
		Thread miEst = new Thread(miEstHi);
		miArrEs.add(miEst.getState());		// Aqui anade el estado NEW

		miEst.start();
		// Estados por los que va pasando antes de terminar
		while (miEst.getState() != Thread.State.TERMINATED)
		{
			if (! miArrEs.contains(miEst.getState()))
			{
				miArrEs.add(miEst.getState());
```

```
                        }
                }

                if (! miArrEs.contains(miEst.getState()))  // Aqui pone el estado TERMINATED
                {
                        miArrEs.add(miEst.getState());
                }

                for (Thread.State estado : miArrEs)
                {
                        System.out.println(estado);
                }
        }

        @Override
        public void run() {
                try {
                        Thread.sleep(100);              // Ponerlo a dormir anade el estado TIMED_WAITING
                } catch (InterruptedException e) {
                        e.printStackTrace();
                }
        }
}
```

Timer basado en hilos

```
// Con ps -ef se puede comprobar que los timer son hilos, no procesos. El SO solo activa un proceso
import java.util.Timer;
public class T2Timer {
        public static void main (String [] args)
        {
                Timer miTimer =  new Timer();
                // miTimer.schedule(new T2TimerEjemplo(),1000,2000);     // Inicia al segundo, se repite cada dos
                // miTimer.schedule(new T2TimerEjemplo(),1000);          // Una ejecución despues de 1 segundo
                miTimer.schedule(new T2TimerTarea(),1000,2000);          // Llama a otra tarea

                // Dejamos un tiempo para que haya varias ejecuciones antes del cancel (parada del timer)
                try {   Thread.sleep(6000);}
                catch (InterruptedException e) {e.printStackTrace();}

                // Para el timer. Si no paras el timer, aunque termine main, sigue activo. Comprobar tambien con ps
-ef
                miTimer.cancel();

                System.out.println("Parado el timer");

                // Dejamos un tiempo para comprobar que el timer ya no esta activo
                try {   Thread.sleep(5000);}
                catch (InterruptedException e) {e.printStackTrace();}

                System.out.println("Finaliza el programa");
        }
}
-------------------------------------------------------
import java.util.TimerTask;
public class T2TimerEjemplo extends TimerTask {
        @Override
        public void run() {
                System.out.println("Saludos 2-DAM");
        }
}
-------------------------------------------------------
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.TimerTask;
public class T2TimerTarea extends TimerTask{
        @Override
        public void run() {
                DateFormat dateF = new SimpleDateFormat("HH:mm:ss");
                Date miDate = new Date();
                System.out.println(dateF.format(miDate));
        }
}
```

Join, Yield, Yield equivalente a Thread.sleep(0)

```
// Probar.
//  - Iniciar dos hilos con join. Probar dif. vueltas en bucle de los hijos.
//  - Con muchas hay cambios de procesos y Con join el mensaje Termina programa sale al final
//  - Iniciar los dos hilos, sin join y probar con muchas vueltas. Termina programa sale entre medias de los mensajes
//
//   - En hijouno, meter yield. Hace que se salga y da paso al hijo dos, al padre o podría volver a ser planificado el hilo 1

//                      Thread.sleep(0); es equivalente a yield
public class T2JoinYield {
        public static void main (String [] args)
        {
                T2JoinYieldH1 hijoUno = new T2JoinYieldH1("Hijo uno");
                T2JoinYieldH2 hijoDos = new T2JoinYieldH2("Hijo dos");
                hijoUno.start();
                hijoDos.start();

                try { hijoUno.join();}
                catch(InterruptedException e) {         e.printStackTrace();}
                try { hijoDos.join(); }
                catch (InterruptedException e) { e.printStackTrace(); }

                System.out.println("Termina programa");
        }
}
--------------------------------
public class T2JoinYieldH1 extends Thread {
        String sNombre;
        public T2JoinYieldH1(String sNombre)
        {
                this.sNombre = sNombre;
        }
        @Override
        public void run ()
        {
                System.out.println("Inciando "+ sNombre);
                for (int i = 0; i < 500; i++)
                {
                        if (i==5) {
                                // this.yield();
                                try { Thread.sleep(0); }
                                catch (InterruptedException e) { e.printStackTrace(); }
                        }
                        System.out.println(sNombre + " Pasada: " + i);
                }
                System.out.println("Terminando "+ sNombre);
        }
}
public class T2JoinYieldH2 extends Thread {
        String sNombre;
        public T2JoinYieldH2(String sNombre)
        {
                this.sNombre = sNombre;
        }
        @Override
        public void run ()
        {
                System.out.println("Inciando "+ sNombre);

                for (int i = 0; i < 500; i++)
                {
                                System.out.println(sNombre + " Pasada: " + i);
                }

                System.out.println("Terminando "+ sNombre);
        }
}
```

Uso de Monitor. Exclusión mutua con Synchronized en método

```
// En VirtualBox. Funciona indicando que tenga varios procesadores en Configuracion/Sistema/Procesador/CPU
// Tb se puede probar con java -jar FicheroJarExportado.jar
// Probar con el metodo T2MonitorCom.IncrementaCon con la etiqueta synchronized y sin ella
public class T2Monitor {
	public static void main(String[] args) {
		T2MonitorComp miMonitor = new T2MonitorComp();

		T2MonitorHilo miHiloUno = new T2MonitorHilo(miMonitor, "Hilo Uno");
		T2MonitorHilo miHiloDos = new T2MonitorHilo(miMonitor, "Hilo Dos");
		T2MonitorHilo miHiloTres = new T2MonitorHilo(miMonitor, "Hilo Tres");
		T2MonitorHilo miHiloCuatro = new T2MonitorHilo(miMonitor, "Hilo Cuatro");
		T2MonitorHilo miHiloCinco = new T2MonitorHilo(miMonitor, "Hilo Cinco");
		T2MonitorHilo miHiloSeis = new T2MonitorHilo(miMonitor, "Hilo Seis");

		miHiloUno.start();
		miHiloDos.start();
		miHiloTres.start();
		miHiloCuatro.start();
		miHiloCinco.start();
		miHiloSeis.start();

		try {
			miHiloUno.join();
			miHiloDos.join();
			miHiloTres.join();
			miHiloCuatro.join();
			miHiloCinco.join();
			miHiloSeis.join();
		} catch (InterruptedException e) {	e.printStackTrace();}

		System.out.println(miMonitor.getContador());
	}
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
public class T2MonitorHilo extends Thread {

	T2MonitorComp zonaComp;
	String sNombre;

	public T2MonitorHilo (T2MonitorComp zonaComp, String sNombre)
	{
		this.zonaComp = zonaComp;
		this.sNombre = sNombre;
	}

	@Override
	public void run() {
		for (int i = 0 ; i < 600000; i++)
		{
			zonaComp.IncrementaCon();
			System.out.println("Soy " + sNombre + ": " + zonaComp.getContador());
		}
	}
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
// Usa syncrhonized asociado a un metodo. Bloquea todo el objeto de la clase
public class T2MonitorComp {
	private int iContador = 0;

	public synchronized void IncrementaCon ()
	// public void IncrementaCon ()
	{
		iContador = iContador + 1;
	}

	public int getContador ()
	{
		return iContador;
	}
}
```

Uso de Monitor. Exclusión mutua con Synchronized por bloque

class T2Monitor y class T2MonitorComp igual que en el ejercicio anterior

```
public class T2MonitorHilo extends Thread {

	T2MonitorComp zonaComp;
	String sNombre;

	public T2MonitorHilo (T2MonitorComp zonaComp, String sNombre)
	{
		this.zonaComp = zonaComp;
		this.sNombre = sNombre;
	}

	@Override
	public void run() {
		for (int i = 0 ; i < 200000; i++)
		{
			synchronized (zonaComp) {
				zonaComp.IncrementaCon();
				System.out.println("Soy " + sNombre + ": " + zonaComp.getContador());
			}
		}
	}
}
```

Uso de Monitor. Dos hilos bloqueados por mal uso de Synchronized

Programa ejemplo de interbloqueo o deadlock. En este ejemplo tenemos dos hilos, cada uno precisa permiso "en exclusiva" de escritura en dos archivos distintos.
"Hilo 1" podría abrir "Archivo A" de forma exclusiva e "Hilo 2" podría hacer lo mismo con "Archivo B".
Estando "Hilo 1" utilizando "Archivo A" necesita, a la misma vez, acceso "en exclusiva" sobre "Archivo B" y estando a su vez "Hilo 2" haciendo uso de "Archivo B", éste hilo necesita acceso exclusivo sobre "Archivo A".
En este escenario ambos hilos se obstaculizan entre sí y se bloquean. En este momento podemos decir que se habrá producido un interbloqueo.

```
public class Main {

   static ArchivoA a = new ArchivoA(); // Se crean dos recursos de tipo "Archivo",
   static ArchivoB b = new ArchivoB(); // uno de cada tipo, o sea "Archivo A" y "Archivo B"

   public static void main(String args[]) {

      Hilo1 hilo1 = new Hilo1(a, b);  // creación de hilos que serán los
      Hilo2 hilo2 = new Hilo2(a, b);  // responsables del interbloqueo

      hilo1.start();              // ejecución de lógica que invoca métodos
      hilo2.start();              // del tipo 'synchronized'
   }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

public class Hilo1 extends Thread {
   ArchivoA a; // declaración de objetos, uno
   ArchivoB b; // de cada tipo de archivo

   Hilo1(ArchivoA a, ArchivoB b) {
      this.a = a;
      this.b = b;
   }

   /**
    * ********************************************************************************
    * Lógica asociada a la ejecución del hilo: capturar el recurso 'ArchivoA'
    */
   @Override
   public void run() {
      a.metodoA(b); //el hilo captura el recurso 'ArchivoA'
   }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

```
public class Hilo2 extends Thread {

    ArchivoA a; // declaración de objetos, uno
    ArchivoB b; // de cada tipo de archivo

    Hilo2(ArchivoA a, ArchivoB b) {
        this.a = a;
        this.b = b;
    }

    /**
     * ********************************************************************************
     * Lógica asociada a la ejecución del hilo: capturar el recurso 'ArchivoA'
     */
    @Override
    public void run() {
        b.metodoB(a); //el hilo captura el recurso 'ArchivoA'
    }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

public class ArchivoA {

    /**
     * ********************************************************************************
     * métodoA: simula el acceso al recurso ArchivoA con exclusión mutua. Duerme
     * al hilo y después invoca al método que le permitirá o intentará el acceso
     * a ArchivoB
     *
     * @param b: ArchivoB
     */
    public synchronized void metodoA(ArchivoB b) {

        String name = Thread.currentThread().getName(); // imprime en la salida
                                            // el nombre del hilo que
                                            // ejecuta el método
        System.out.println("Hilo " + name + " entra en ArchivoA");

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Excepción: "+e.getMessage());
        }

        System.out.println("Hilo " + name + " intentando acceder a fichero_B");
        b.metodoB(this);
    }

    public synchronized void syn2() {

        System.out.println("Dentro de A.syn2");
    }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

public class ArchivoB {

    /**
     * ********************************************************************************
     * métodoA: simula el acceso al recurso ArchivoA con exclusión mutua. Duerme
     * al hilo y después invoca al método que le permitirá o intentará el acceso
     * a ArchivoB
     *
     * @param a: ArchivoA
     */
    public synchronized void metodoB(ArchivoA a) {

        String name = Thread.currentThread().getName(); // imprime en la salida
                                            // el nombre del hilo que
                                            // ejecuta el método
        System.out.println("Hilo " + name + " entra en ArchivoB");

        try {
```

```
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Excepción: "+e.getMessage());
        }

        System.out.println("Hilo " + name + " intentando acceder a ArchivoA");
        a.metodoA(this);
    }
}
```

Captura de excepciones centralizada desde diferentes hilos

La gestión de errors resulta más complicada que programas secuenciales al tener varias vias de código en ejeución en un mismo programa. La clase UnCaugthException nos ayuda a poner en común la captura de excepciones vengan de unos hilos u otros.

```
public class Main {

    public static void main(String[] args) {

        // creación de 5 hilos
        Hilo hilo1 = new Hilo("hilo1");
        Hilo hilo2 = new Hilo("hilo2");
        Hilo hilo3 = new Hilo("hilo3");
        Hilo hilo4 = new Hilo("hilo4");
        Hilo hilo5 = new Hilo("hilo5");

        ManejadorExcepciones manejador = new ManejadorExcepciones(); // se crea
                                        // el manejador de excepciones

        hilo1.setUncaughtExceptionHandler(manejador);   // se asocia a cada hilo
        hilo2.setUncaughtExceptionHandler(manejador);   // el manejador de
        hilo3.setUncaughtExceptionHandler(manejador);   // excepciones creado
        hilo4.setUncaughtExceptionHandler(manejador);
        hilo5.setUncaughtExceptionHandler(manejador);

        hilo1.start();  // se inicia la ejecución de los hilos
        hilo2.start();
        hilo3.start();
        hilo4.start();
        hilo5.start();
    }
}
```

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

```
import java.util.Random;

public class Hilo extends Thread {

    public Hilo(String nombre) {
        super(nombre);
    }

    // lógica asociada al hilo
    public void run() {
        Random numero = new Random(); // creo un objeto que modela la aleatoriedad
        int division = 100 / numero.nextInt(4); // construye el valor aleatorio
                        // dividiendo que un valor que oscile entre 0 y 4
        System.out.println("Division: " + division);    // muestra en la salida
                            // estándar el resultado final obtenido
    }
}
```

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

```
/**
 * Clase que representa el manejador de excepciones que será utilizado en
 * el aplicativo
*/
public class ManejadorExcepciones implements Thread.UncaughtExceptionHandler {
```

```
    @Override
    public void uncaughtException(Thread hilo, Throwable excepcion) { // redefine
                                    // el método 'uncaughtException'
        System.out.println("El hilo '" + hilo.getName() + "' ha devuelto la siguiente"
                + " excepción:");
            excepcion.printStackTrace(); //muestra en consola la pila de llamadas
    }
}
```

Monitor – Productor-Consumidor. Wait, notify, notifyall, interrupt

```
// Crea el monitor, genera dos productores y dos consumidores. Los cuatro son hilos.
// Pone en marcha tanto los productores, como los consumidores.
// Lanza una interrupcion a los hilos para que finalicen
import java.util.Scanner;

public class T2ProdConUno {
        Scanner miScan = new Scanner (System.in);

        public static void main (String [] args)
        {
                Monitor miMon = new Monitor();                  // Alta del monitor
                // Crea los cuatro hilos (dos productores, dos consuminodres)
                Productor prodUno = new Productor (miMon,"PROC-1");
                Consumidor conUno = new Consumidor(miMon,"CON-1");
                Productor prodDos = new Productor (miMon, "PROC-2");
                Consumidor conDos = new Consumidor(miMon, "CON-2");
                Thread hiloProdUno = new Thread(prodUno);
                Thread hiloConUno = new Thread(conUno);
                Thread hiloProdDos = new Thread(prodDos);
                Thread hiloConDos = new Thread(conDos);
                // Inicia los hilos.
                hiloProdUno.start();
                hiloConUno.start();
                hiloProdDos.start();
                hiloConDos.start();

                try {Thread.sleep(20000);}
                catch (InterruptedException e) { e.printStackTrace();}

                hiloProdUno.interrupt();
                hiloConUno.interrupt();
                hiloProdDos.interrupt();
                hiloConDos.interrupt();
        }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

// Monitor. Array que gestiona el acceso con exclusion mutua al array.
// Utiliza synchronized, wait y notifyall
import java.time.LocalDateTime;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;

public class Monitor {
        final int REG = 3;
        int iNumAlm = 0;

        // Version 1 -- INI con array
//      String miArr[] = new String[REG];
        // Version 1 -- FIN

        // Version 2 -- INI con coleccion
        Set <String> miCol = new TreeSet <String>();
        // Version 2 -- FIN

        public synchronized void Producir()
        {
                while (iNumAlm == REG) {
                        try {
                                wait();
```

```
                        } catch (InterruptedException e) {          e.printStackTrace();}
                }
                        // Anadir fecha al buffer
                        LocalDateTime ldt = LocalDateTime.now();

                        // Version 1 -- INI con array
//                      miArr[iNumAlm] = ldt.getHour() + ":" + ldt.getMinute() + ":" + ldt.getSecond();
//                      System.out.println("Produce a las " + miArr[iNumAlm]);
                        // Version 1 -- FIN

                        // Version 2 -- INI con coleccion
                        String sTiempo = ldt.getHour() + ":" + ldt.getMinute() + ":" + ldt.getSecond();
                        miCol.add(sTiempo);
                        System.out.println("Produce a las " + sTiempo);
                        // Version 2 -- FIN

                        iNumAlm++;
                        notifyAll();
        }

        public synchronized void Consumir()
        {
                while (iNumAlm == 0) {
                        try {
                                wait();
                        } catch (InterruptedException e) {          }
                }
                //  Leer del buffer
                iNumAlm--;
                // Version 1 -- INI con array
//              System.out.println("Consume a las " + miArr[iNumAlm]);
                // Version 1 -- FIN

                // Version 2 -- INI con coleccion
                Iterator miIter = miCol.iterator();
                System.out.println("Consume a las " + miIter.next());
                miIter.remove();
                // Version 2 -- FIN

                notifyAll();
        }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

// Hilo productor. Llama al metodo del monitor que genera una nueva entrada
// Bucle infinito, termina con la llegada de una interrupcion
public class Productor  implements Runnable {

        Monitor miMon;
        String sNom;

        public Productor (Monitor mon, String nombre) {
                miMon = mon;
                sNom = nombre;
        }

        @Override
        public void run() {
                // Bucle infinito, sale por el catch por la llegada de una interrupcion
                 while (true)
                 {
                         System.out.println("Entra Producir " + sNom);
                         miMon.Producir();
                         try {
                                 Thread.sleep(3000 + (int)(Math.random() * 5000));
                        } catch (InterruptedException e) {
                                 break;
                        }
                 }
                 System.out.println("Terminando productor " + sNom);
        }
}
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

// Hilo consumidor. Llama al metodo del monitor que gasta una entrada de las existentes
```

```
// Bucle infinito, termina con la llegada de una interrupcion
public class Consumidor   implements Runnable {

	Monitor miMon;
	String sNom;

	public Consumidor (Monitor mon, String nombre) {
		miMon = mon;
		sNom = nombre;
	}

	@Override
	public void run() {
		// Bucle infinito, sale por el catch por la llegada de una interrupcion
		while (true)
		 {
			System.out.println("Entra Consumir " + sNom);

			miMon.Consumir();
			try {
				Thread.sleep(3000 + (int)(Math.random() * 5000));
			} catch (InterruptedException e) {
				break;
			}
		 }
		 System.out.println("Terminando consumidor " + sNom);
	}
}
```

Monitor – Productor-Consumidor. semaforo, interrupt

```
// Crea el monitor gestionado con semaforos, genera dos productores y dos consumidores. Los cuatro son hilos.
// Pone en marcha tanto los productores, como los consumidores.
// Lanza una interrupcion a los hilos para que finalicen
public class T2ProdConUno {
	public static void main (String [] args)
	{
		Monitor miMon = new Monitor();                                  // Alta del monitor
		// Crea los cuatro hilos (dos productores, dos consuminodres)
		Productor prodUno = new Productor (miMon,"PROC-1");
		Consumidor conUno = new Consumidor(miMon,"CON-1");
		Productor prodDos = new Productor (miMon, "PROC-2");
		Consumidor conDos = new Consumidor(miMon, "CON-2");
		Thread hiloProdUno = new Thread(prodUno);
		Thread hiloConUno = new Thread(conUno);
		Thread hiloProdDos = new Thread(prodDos);
		Thread hiloConDos = new Thread(conDos);
		// Inicia los hilos.
		hiloProdUno.start();
		hiloConUno.start();
		hiloProdDos.start();
		hiloConDos.start();

		try {
			Thread.sleep(20000);
		} catch (InterruptedException e) {	e.printStackTrace();}

		hiloProdUno.interrupt();
		hiloConUno.interrupt();
		hiloProdDos.interrupt();
		hiloConDos.interrupt();
	}
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

// Monitor. Utiliza semaforo
import java.time.LocalDateTime;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.Semaphore;
```

```
public class Monitor {
	final int REG = 3;
	Semaphore almacen = new Semaphore (REG, true);

	int iNumAlm = 0;
	Set <String> miCol = new TreeSet <String>();

	public synchronized void Producir()
	{
		try {
			almacen.acquire();
		} catch (InterruptedException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}

			// Anadir fecha al buffer
			LocalDateTime ldt = LocalDateTime.now();

			String sTiempo = ldt.getHour() + ":" + ldt.getMinute() + ":" + ldt.getSecond();
			miCol.add(sTiempo);
			System.out.println("Produce a las " + sTiempo);

			iNumAlm++;
			almacen.release();
	}

	public synchronized void Consumir()
	{
		try {
			almacen.acquire();
		} catch (InterruptedException e) {
			// TODO Auto-generated catch block
			e.printStackTrace();
		}

		if(!miCol.isEmpty() )
		{
			//  Leer del buffer
			iNumAlm--;
			Iterator miIter = miCol.iterator();
			System.out.println("Consume a las " + miIter.next());
			miIter.remove();
		}
		almacen.release();
	}
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

// Hilo productor. Llama al metodo del monitor que genera una nueva entrada
// Bucle infinito, termina con la llegada de una interrupcion
public class Productor  implements Runnable {

	Monitor miMon;
	String sNom;

	public Productor (Monitor mon, String nombre) {
		miMon = mon;
		sNom = nombre;
	}

	@Override
	public void run() {
		// Bucle infinito, sale por el catch por la llegada de una interrupcion
		 while (true)
		 {
			 System.out.println("Entra Producir " + sNom);
			 miMon.Producir();
			 try {
				Thread.sleep(3000 + (int)(Math.random() * 5000));
			 } catch (InterruptedException e) {
				break;
			 }
		 }
```

```
                System.out.println("Terminando productor " + sNom);
        }
}
```

++

```
// Hilo consumidor. Llama al metodo del monitor que gasta una entrada de las existentes
// Bucle infinito, termina con la llegada de una interrupcion
public class Consumidor   implements Runnable {

        Monitor miMon;
        String sNom;

        public Consumidor (Monitor mon, String nombre) {
                miMon = mon;
                sNom = nombre;
        }

        @Override
        public void run() {
                // Bucle infinito, sale por el catch por la llegada de una interrupcion
                while (true)
                {
                        System.out.println("Entra Consumir " + sNom);

                        miMon.Consumir();
                        try {
                                Thread.sleep(3000 + (int)(Math.random() * 5000));
                        } catch (InterruptedException e) {
                                break;
                        }
                }
                System.out.println("Terminando consumidor " + sNom);
        }
}
```

Clase Executor

```
// Clase Executors. Alternativa a Timer y disponible en java.util.concurrent
// Solo alguna altermanativa de la clase
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class T2Executor {
        public static void main (String [] args)
        {
                // Lanza un solo servicio
                // ScheduledExecutorService ses = Executors.newSingleThreadScheduledExecutor();
                // ses.scheduleAtFixedRate(new T2ExecutorTarea(), 1, 2, TimeUnit.SECONDS);

                /*
                // Gestiona multiples procesos en ejecucion. Hace una cola de procesos
                // Solo tres procesos simultaneos
                ExecutorService sesPool = Executors.newFixedThreadPool(3);
                ThreadPoolExecutor miTPE = (ThreadPoolExecutor) sesPool;
                sesPool.submit(new T2ExecutorTarea());
                sesPool.submit(new T2ExecutorTareaDos());
                sesPool.submit(new T2ExecutorTareaDos());
                sesPool.submit(new T2ExecutorTarea());
                sesPool.submit(new T2ExecutorTareaDos());
                sesPool.submit(new T2ExecutorTarea());
                sesPool.submit(new T2ExecutorTarea());
                 */
        }
}
```

++

```
public class T2ExecutorTarea implements Runnable {

	@Override
	public void run() {
		System.out.println("Probando Executors");
	}
}

----
public class T2ExecutorTareaDos implements Runnable {

	@Override
	public void run() {

		while (true)			// Para que no termine y comprobar que no se activan los siguientes
procesos
		{
			System.out.println("Probando Executors 2");
			try {
				Thread.sleep(10000);
			} catch (InterruptedException e) {
				// TODO Auto-generated catch block
				e.printStackTrace();
			}
		}
	}
}
```

Clase Exchanger

Intercambia un objeto entre hilos. En el constructor de Exchanger se puede indicar un timeout. El primer hilo a invocar Exchanger queda a la espera bloqueado de que el método sea llamado también por el otro hilo. Hace en bucle intercambios de objetos (cadenas de caracteres en este ejercicio) durante un segundo.

```
import java.util.concurrent.Exchanger;

/**
 * El hilo principal de esta aplicación crea dos nuevos hilos con un punto de
 * sincronización tipo 'Exchanger'. El hilo denominado 'hilo productor' se encarga
 * de rellenar una cadena o lista de diez caracteres
 * El hilo denominado 'hilo consumidor' imprime esa lista
 * Dejaremos que ambos hilos funcionen alrededor de un segundo antes de proceder
 * a deternerlos
 *
 * @author IMCG
 * @author Alejandro Leal
 */
public class Main {

    public static void main(String[] args) {
        Exchanger<String> intercambiador = new Exchanger<>(); // elemento de
                                        // intercambio para objetos
                                        // de la clase 'String'
        HiloProductor productor = new HiloProductor(intercambiador); // creación
                                                // del hilo
                                                // productor
        productor.start();  // se inicia la ejecución del hilo productor

        (new HiloConsumidor(intercambiador)).start();   // creación del hilo
                                        // consumidor
        try {
            Thread.sleep(1000); // duerme el hilo principal durante un segundo
                        // para proporcionar la oportunidad de que los
                        // hilos intercambien los objetos
                        // Esto quiere decir que los hilos implicados en
                        // este escenario ('productor' y 'consumidor')
                        // dispondrán de un tiempo total global de 1
                        // segundo de ejecución
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());    // se muestra el error por la
                                        // salida estándar
        }
```

```
        productor.parada(); // detención del hilo productor (el cual enviará, a
                            // su vez, dicha señal al hilo consumidor en el
                            // turno de intercambio
    }
}

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

import java.util.concurrent.Exchanger;

/**
 * Implementación de la clase que encapsula al hilo de tipo 'consumidor'
 * @author Alejandro Leal Cruz
 */
public class HiloConsumidor extends Thread {

    final Exchanger<String> intercambiador; // elemento de intercambio para objetos
                                            // de la clase 'String'
    String cadena; // cadena recibida en el último intercambio

    /**
     * Constructor de hilo de tipo 'consumidor' que recibe como parámetro un
     * intercambiador decadena
     *
     * @param intercambiadorHilo: elemento de intercambio
     */
    HiloConsumidor(Exchanger<String> intercambiadorHilo) {
        intercambiador = intercambiadorHilo;
    }

    /**
     * Mientras que el hilo 'productor' no envía la cadena de intercambio vacía,
     * esa es la señal de parada, se imprime en la salida la cadena de 10
     * caracteres recibida en el último intercambio con el hilo 'productor'
     * Cuando ello suceda se intercambiará con el hilo 'productor' una cadena
     * vacía por la nueva recibida que sí contiene caracteres
     */
    @Override
    public void run() {
        while (cadena == null || cadena.length() > 0) { //mientras que no se haya
                                // realizado un intercambio, o se haya
                                // realizado pero la cadena no está vacía
            try {
                cadena = intercambiador.exchange(""); // invoca el método 'exchange',
                        // con el argumento cadena vacía (""), para intercambiar
                        // con hilo 'productor' una cadena vacía por otra que
                        // esté rellena (esto bloquea la ejecución de hilo
                        // 'consumidor' hasta que hilo 'productor' esté listo
                        // para realizar el intercambio)

                if (cadena.length() > 0) { //si en el intercambio no se recibió
                        // la señal de parada entonces se imprime la cadena
                        // en la salida estándar:
                    System.out.println("Hilo 'consumidor' contiene " + cadena +
                        ", mientras hilo 'productor' genera el siguiente "
                            + "paquete de información...");
                }
            } catch (InterruptedException ex) {
                System.out.println(ex.getMessage());
            }
        }
    }
}
```